Day 5: Functions, Scopes & Regular Expressions

Welcome to Day 5! Today we will explore:

1. **Functions and Abstraction**: Organizing and modularizing code.
2. **Scoping Rules**: How variable lookups work under the LEGB rule.
3. **Anonymous (Lambda) Functions**: Creating light, one-line functions.
4. **Built-in Helpers**: Inspecting and manipulating data with standard functions.
5. **Regular Expressions (RegEx)**: Pattern matching and string manipulation.

Part 1: Functions & Abstraction

1. Defining and Calling Functions

A **function** is a reusable block of organized code used to perform a single, related action. Functions provide better modularity for your application and a high degree of code reusing.

```
# Defining a simple function
def greet_student(name):
    """Docstring explaining the function's purpose: greet a student."""
    return f"Welcome, {name}, to CDAC PGCP-AI!"

# Calling the function
message = greet_student("Arham")
print(message)  # Output: Welcome, Arham, to CDAC PGCP-AI!
```

2. Argument Passing Mechanics

Python offers extremely flexible ways to pass arguments to functions.

A. Positional and Keyword Arguments

- **Positional Arguments**: Assigned based on their position/order in the call.
- **Keyword Arguments**: Assigned by specifying parameter names during the call, allowing you to pass them in any order.

```python
def describe_pet(animal_type, pet_name):
    print(f"My {animal_type}'s name is {pet_name}.")

# Positional call
describe_pet("Hamster", "Harry")  # Output: My Hamster's name is Harry.

# Keyword call (order doesn't matter)
describe_pet(pet_name="Bruno", animal_type="Dog")  # Output: My Dog's name
is Bruno.
```

B. Default Parameter Values

Parameters can have default values. If a value is not supplied during execution, the default is used.

```python
def make_coffee(size, flavor="Regular"):
    print(f"Serving a {size} cup of {flavor} coffee.")

make_coffee("Large")             # Output: Serving a Large cup of Regular
coffee.
make_coffee("Medium", "Vanilla") # Output: Serving a Medium cup of Vanilla
coffee.
```

> [!IMPORTANT] Non-default parameters must always be declared **before** default parameters in the function definition. `def func(a=10, b):` is syntax error.

C. Arbitrary Arguments: `*args` and `**kwargs`

- `*args`: Collects extra positional arguments as a **tuple**.
- `**kwargs`: Collects extra keyword arguments as a **dictionary**.

```python
def report_achievements(student_name, *subjects, **details):
    print(f"Student: {student_name}")
    print(f"Enrolled in: {subjects}")
    print(f"Metadata:")
    for key, val in details.items():
        print(f" - {key}: {val}")

report_achievements("Lisa", "Python", "AI Basics", batch="August 2026",
id="A104")
# Output:
# Student: Lisa
# Enrolled in: ('Python', 'AI Basics')
# Metadata:
#  - batch: August 2026
#  - id: A104
```

D. Keyword-Only and Positional-Only Arguments

Introduced in modern Python:

- `/`: Denotes parameters to its left must be **positional-only**.
- `*`: Denotes parameters to its right must be **keyword-only**.

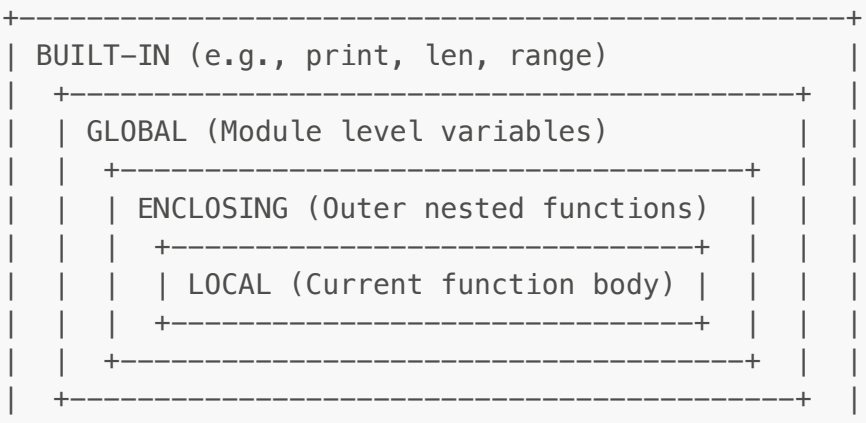
```
def strict_function(pos_only, /, standard, *, kw_only):
    print(pos_only, standard, kw_only)

# Valid call
strict_function(10, "hello", kw_only="world")

# Invalid calls (will raise TypeError)
# strict_function(pos_only=10, standard="hello", kw_only="world")
# strict_function(10, "hello", "world")
```

Part 2: Scoping Rules (LEGB Rule)

Python looks up variables in a specific order: **L**ocal → **E**nclosing → **G**lobal → **B**uilt-in.



1. Variables and Boundaries

- **Local**: Variables created inside the executing function.
- **Enclosing**: Variables inside outer scopes of nested functions.
- **Global**: Variables declared at the top-level of a module.
- **Built-in**: Names preloaded by Python (like `print()`, `ValueError`).

2. The `global` Keyword

To modify a variable defined at the module-level from inside a function, declare it as `global`.

```
count = 10  # Global variable

def increment_global():
    global count
    count += 1
    print("Inside function:", count)

increment_global() # Output: Inside function: 11
print("Global scope:", count) # Output: Global scope: 11
```

3. The `nonlocal` Keyword

In nested functions, to modify a variable in the immediate outer (enclosing) scope, declare it as `nonlocal`.

```
def outer_counter():
    step = 0  # Enclosing scope variable

    def inner():
        nonlocal step
        step += 1
        return step

    return inner

counter = outer_counter()
print(counter())  # Output: 1
print(counter())  # Output: 2
```

Part 3: Anonymous (Lambda) Functions

A **lambda function** is a small, anonymous function that can have any number of arguments but only a **single expression**.

Syntax:

```
lambda arguments: expression
```

```
# Simple addition lambda
add = lambda x, y: x + y
print(add(5, 7))  # Output: 12

# Commonly used with map, filter, and sorted:
numbers = [1, 2, 3, 4, 5, 6]
```

```python
# 1. Filter: extract even values
evens = list(filter(lambda x: x % 2 == 0, numbers))
print("Evens:", evens)  # Output: [2, 4, 6]

# 2. Map: square the list
squares = list(map(lambda x: x**2, numbers))
print("Squares:", squares)  # Output: [1, 4, 9, 16, 25, 36]

# 3. Sorted: sorting tuples by second value
points = [(1, 9), (5, 2), (3, 7)]
points_sorted = sorted(points, key=lambda point: point[1])
print("Sorted Points:", points_sorted)  # Output: [(5, 2), (3, 7), (1, 9)]
```

Part 4: Built-in Helper Functions

Python has useful built-in inspection helpers:

- `type(obj)`: Returns the type of `obj`.
- `id(obj)`: Returns the memory identity of `obj`.
- `dir(obj)`: Lists valid attributes/methods available on `obj`.
- `enumerate(iterable)`: Returns an iterator yielding tuple pairs: `(index, item)`.
- `zip(*iterables)`: Aggregates elements from multiple iterables into tuples.

```python
# Enumeration demo
names = ["Alice", "Bob"]
for idx, name in enumerate(names, start=1):
    print(f"{idx}: {name}")

# Zip demo
scores = [85, 92]
zipped = dict(zip(names, scores))
print("Zipped Dict:", zipped)  # Output: {'Alice': 85, 'Bob': 92}
```

Part 5: Regular Expressions (RegEx)

Regular expressions are patterns used to match and extract character combinations in strings. In Python, use the `re` module.

1. Key Meta-characters

- `\d`: Matches any decimal digit (equivalent to `[0-9]`).
- `\w`: Matches alphanumeric characters and underscores (`[a-zA-Z0-9_]`).
- `\s`: Matches whitespace characters (spaces, tabs, newlines).
- `+`: Matches 1 or more repetitions of the preceding pattern.
- `*`: Matches 0 or more repetitions of the preceding pattern.

- `?`: Matches 0 or 1 repetition of the preceding pattern.
- `^` / `$`: Matches the start / end of a string.
- `.`: Matches any character except a newline.

2. Core `re` Module Functions

A. Finding Matches: `re.search()` vs `re.match()`

- `re.match()`: Checks for a match **only at the beginning** of the string.
- `re.search()`: Scans the **entire string** for a match.

```
import re

text = "CDAC acts Bangalore"

# Match checks only the beginning
match_res = re.match(r"acts", text)
print("Match found:", match_res)  # Output: None

# Search checks the entire string
search_res = re.search(r"acts", text)
print("Search found:", search_res.group())  # Output: acts
```

B. Getting Multiple Matches: `re.findall()` & `re.finditer()`

- `re.findall(pattern, string)`: Returns all non-overlapping matches as a list of strings.
- `re.finditer(pattern, string)`: Returns an iterator yielding match objects.

```
numbers_text = "Today is 28th, temperature is 26 degrees, speed limit is
60."
digits = re.findall(r"\d+", numbers_text)
print("Digits:", digits)  # Output: ['28', '26', '60']
```

C. Substituting Patterns: `re.sub()`

Replaces occurrences of a pattern with a replacement string.

```
raw_log = "Secret code: 456-789. System OK."
# Mask numeric codes
masked_log = re.sub(r"\d+", "XXX", raw_log)
print("Masked:", masked_log)  # Output: Secret code: XXX-XXX. System OK.
```

3. Capture Groups and Patterns

By surrounding parts of your regex with parentheses `()`, you define **capture groups** to extract specific subsets of matches.

```
email = "info_office@cdac.in"
pattern = r"^([a-z0-9._]+)@([a-z0-9.-]+)$"

match = re.search(pattern, email)
if match:
    # group(0) returns the entire matching string
    print("Full Email:", match.group(0))
    # group(1) returns the first capture group
    print("Username:", match.group(1))  # Output: info_office
    # group(2) returns the second capture group
    print("Domain:", match.group(2))    # Output: cdac.in
```

Practical Examples (Interactive & Runnable)

Example 1: Robust Password Quality Assurer

Uses a RegEx query to check password specifications.

```
import re

def is_strong_password(password):
    # Rule 1: Length >= 8
    if len(password) < 8:
        return False, "Password must be at least 8 characters long."

    # Rule 2: At least one uppercase letter
    if not re.search(r"[A-Z]", password):
        return False, "Password must contain at least one uppercase
letter."

    # Rule 3: At least one lowercase letter
    if not re.search(r"[a-z]", password):
        return False, "Password must contain at least one lowercase
letter."

    # Rule 4: At least one digit
    if not re.search(r"\d", password):
        return False, "Password must contain at least one digit."

    # Rule 5: At least one special symbol
    if not re.search(r"[@#$%&+=!]", password):
        return False, "Password must contain at least one special character
(@#$%&+=!)."

    return True, "Strong password!"
```

```
# Run tests
test_pass = "P@ssw0rd2026"
valid, feedback = is_strong_password(test_pass)
print(f"Password '{test_pass}' check: {feedback}")
# Output: Password 'P@ssw0rd2026' check: Strong password!
```

Example 2: Closure-Based Rate Limiter (Stateful Closure)

Demonstrates scopes, closures, and the `nonlocal` keyword to throttle events.

```
import time

def create_rate_limiter(max_calls, interval_seconds):
    """Creates a throttling closure state machine."""
    call_timestamps = []

    def attempt_execution(task_name):
        nonlocal call_timestamps
        current_time = time.time()

        # Keep only timestamps within the current interval window
        call_timestamps = [t for t in call_timestamps if current_time - t <
interval_seconds]

        if len(call_timestamps) < max_calls:
            call_timestamps.append(current_time)
            print(f"[SUCCESS] Running task: {task_name}. Calls in window:
{len(call_timestamps)}")
            return True
        else:
            print(f"[BLOCKED] Rate limit exceeded for {task_name}. Try
again later.")
            return False

    return attempt_execution

# Run Example
limiter = create_rate_limiter(max_calls=2, interval_seconds=3)
limiter("Download File 1") # Success
limiter("Download File 2") # Success
limiter("Download File 3") # Blocked
```